

Day 5: Control Flow & Lookup Tables

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

if / else if / else

Standard conditional branching. In embedded, use for readability where performance is not critical:

```
if (voltage_mv > 4000U) {
    state = BattState::FULL;
} else if (voltage_mv > 3500U) {
    state = BattState::OK;
} else {
    state = BattState::CRITICAL;
}
```

Embedded tip: Avoid deeply nested if/else in ISRs — flat logic with early returns is faster and more predictable.

switch-case

More efficient than long if/else chains when branching on an integer/enum. The compiler can generate a **jump table** (O(1) lookup) for dense enum values:

```
switch (state) {
case LedState::OFF:      return "OFF";          // No fall-through needed
case LedState::SLOW_BLINK: return "SLOW";
default:                 return "UNKNOWN";      // Always handle default
}
```

Ternary Operator

Compact single-line conditional — useful for simple assignments:

```
const char *status = connected ? "online" : "offline";
uint8_t clamped = (val > 255U) ? 255U : val;
```

Lookup Tables — $O(1)$ vs $O(n)$

A **lookup table** stores precomputed results in an array, indexed directly. This replaces an if/else chain with a single array access:

```
// Without lookup table:  $O(n)$  comparisons
uint8_t get_duty(uint8_t level) {
    if (level == 0) return 0;
    if (level == 1) return 36;
    // ...
}

// With lookup table:  $O(1)$ 
static constexpr uint8_t duty_table[8] = {0, 36, 73, 109, 146, 182, 218, 255};
uint8_t get_duty(uint8_t level) { return duty_table[level]; }
```

When to use: CRC tables, sin/cos tables, PWM duty tables, character encoding tables. Especially valuable in ISRs where execution time must be deterministic.

constexpr Arrays for ROM Storage

Declaring a lookup table as `constexpr` guarantees:

1. Computed at compile time (no runtime initialization cost)
2. Stored in Flash (.rodata) — no RAM consumed
3. Immutable — compiler error if you try to modify it

Avoiding Complex Branches in ISRs

ISRs (Interrupt Service Routines) must execute quickly — ideally under 1-2 microseconds. Rules:

- No `switch` with many cases in an ISR (non-deterministic jump table dispatch)
- Prefer flag-setting + lookup table over if/else chains
- No function calls to unknown-cost functions

Practice Tasks

1. Create a `constexpr` CRC-8 lookup table (256 entries) and verify it's in Flash via the map file
 2. Implement a battery level display (0-5 bars) using a lookup table instead of if/else
 3. Write a state machine with 4 states using `switch` and `enum class` — trace all transitions
 4. Benchmark a 256-entry lookup vs a linear search using `DWT_CYCCNT` cycle counter
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day05
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day05/main.cpp](#)